

Computer System Design & Application

计算机系统设计与应用A

陶伊达 (TAO Yida)

taoyd@sustech.edu.cn

An abstract graphic on the left side of the slide, featuring concentric circles and various digital patterns like binary code and data streams in shades of blue and green.

Lecture 6

- Persistence and Serialization
- Working with Files
- Exception Handling

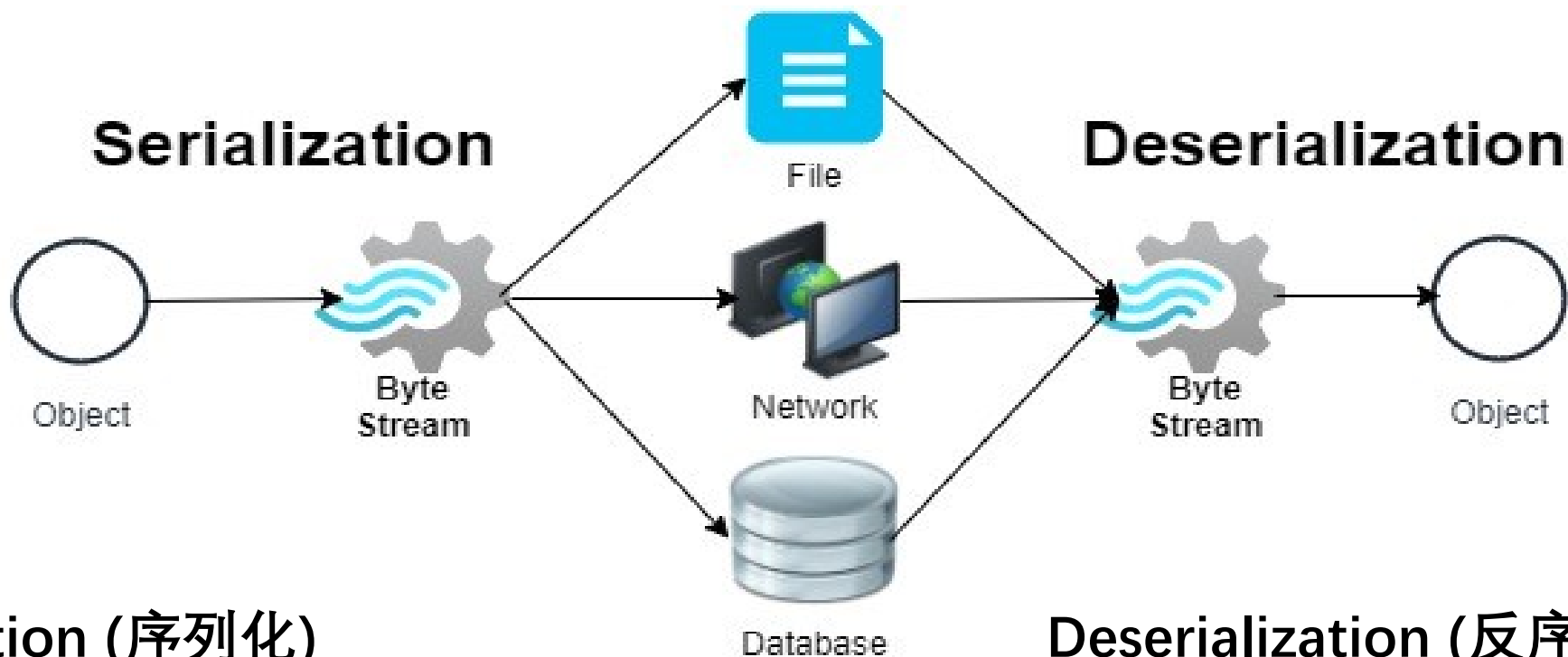
Data Persistence (数据持久化)

- Objects created in Java programs live in memory; they are removed by the garbage collector once they are not used anymore
- What if we want to persist the objects?

Data survives after the process that created it has ended.
Reuse the data without having to executing the program all over again to reach that state.

Data persistence

Store it on a disk, send
it over the network



Serialization (序列化)

Converting the state of an object
into a byte stream

Deserialization (反序列化)

Using the byte stream to recreate
the object in the same state

The Serializable Interface

- Classes need to implement the serializable interface for their instances to be serialized or deserialized
- The serializable interface is called a **marker** interface or **tagging** interface (like putting a tag on the class, so the compiler and JVM, when seeing the tag, knows that the object of the class could be serialized)
 - The serializable interface is an empty interface, without any method or field
 - Classes implementing serializable do not have to implement any methods

```
class Student implements Serializable {  
    String name;  
    String dept;  
    int age;  
  
    public String getName() {  
        return name;  
    }  
  
    public String getDept() {  
        return dept;  
    }  
  
    public int getAge() {  
        return age;  
    }  
  
    public Student(String name, String dept, int age) {  
        this.name = name;  
        this.dept = dept;  
        this.age = age;  
    }  
}
```

```
Student student = new Student("Alice", "CS", 20);
```

```
// Setup where to store the byte stream
```

```
FileOutputStream fos = new FileOutputStream("student.ser");  
ObjectOutputStream oos = new ObjectOutputStream(fos);
```

```
// Serialization
```

```
oos.writeObject(student);
```

```
oos.close();
```

```
//Setup where to read the byte stream
```

```
FileInputStream fis = new FileInputStream("student.ser");  
ObjectInputStream ois = new ObjectInputStream(fis);
```

```
// Deserialization
```

```
Student student2 = (Student) ois.readObject(); // down-casting
```

```
ois.close();
```

```
class Student implements Serializable {
```

```
    String name;
```

```
    String dept;
```

```
    int age;
```

```
    public String getName() {
```

```
        return name;
```

```
    }
```

```
    public String getDept() {
```

```
        return dept;
```

```
    }
```

```
    public int getAge() {
```

```
        return age;
```

```
    }
```

```
    public Student(String name, String dept, int age) {
```

```
        this.name = name;
```

```
        this.dept = dept;
```

```
        this.age = age;
```

```
    }
```

```
}
```



Default Serialization Mechanism

- An ObjectOutputStream looks at all the fields of the objects and save their contents recursively.
- To prevent a field to be serialized, mark it using the transient keyword
- Each object is assigned a unique serial ID to help maintain compatibility when class structure changes

An abstract graphic on the left side of the slide, featuring concentric circles and various digital patterns like binary code and pixelated shapes in shades of blue, green, and white.

Lecture 6

- Persistence and Serialization
- Working with Files
- Exception Handling

Working with Files

- `java.io` (JDK 1.0) is stream-based
 - Focuses on reading and writing sequences of bytes or characters.
 - Good for I/O streams, but weak for managing file systems.
- `java.nio` (JDK 7) is file-system-based
 - Treats the file system itself as the abstraction.: work with paths, attributes, ownership, and directories.
 - Aligns with how OS actually manage storage.



Design Difference

- `java.io` couples data and behavior
 - A `File` object represents a file and provides operations on it.
 - Mixing representation and behavior made it harder to extend.
- `java.nio.file` separates concerns
 - **Path**: represents location or reference to a file or directory ("what you want to act on").
 - **Files**: provides static methods to operate on paths ("how you act on it").



Path

```
import java.nio.file.Path;  
import java.nio.file.Paths;
```

- A Path instance contains the information used to specify the location of a file or directory
- You can easily create a Path object by using one of the following get methods from the Paths helper class

```
Path p1 = Path.of("/home/user/images/pic.png");
```

```
Path p2 = Paths.get("resources");
```

Files

```
import java.nio.file.Files
```

A **utility class** for performing **common file and directory operations** on Path objects.

```
Path path = Path.of("info.txt");

Files.exists(path);
Files.copy(path, Path.of("backup/info.txt"));

Files.readAllLines(path);
BufferedReader reader = Files.newBufferedReader(path)

Files.write(path, List.of("Hello", "World"));
Files.delete(path);
```

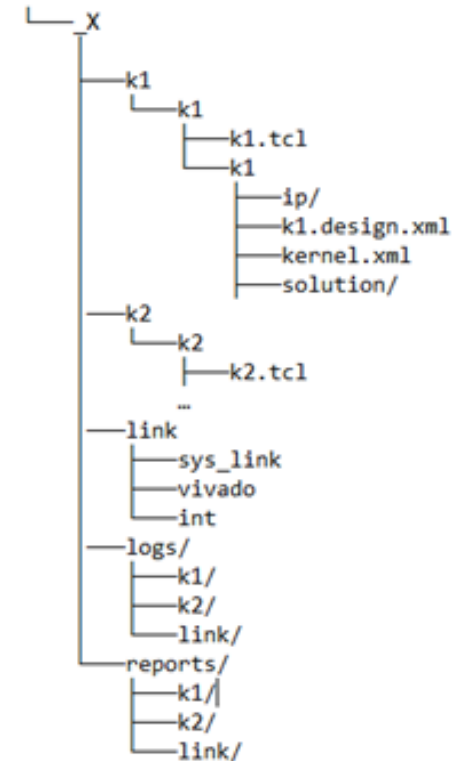
Recursive Directory Traversal

`Files.walk`

Enter all subdirectories in a **depth-first** manner

```
Path dir = Paths.get(workingDir);
```

```
try(Stream<Path> entries = Files.walk(dir)){  
    entries.filter(Files::isRegularFile).forEach(System.out::println);  
}
```



An abstract graphic on the left side of the slide, featuring concentric circles and digital patterns in shades of blue, green, and white, resembling a stylized data visualization or a futuristic interface.

Lecture 6

- Persistence and Serialization
- Working with Files
- Exception Handling

Exception

- An exception indicates that a problem occurs during a program's execution
- An exception disrupts the normal flow of the program

Happy Path

Files are always there
Network is always okay
Memory is always enough
User input is always valid
.....



Unhappy Path

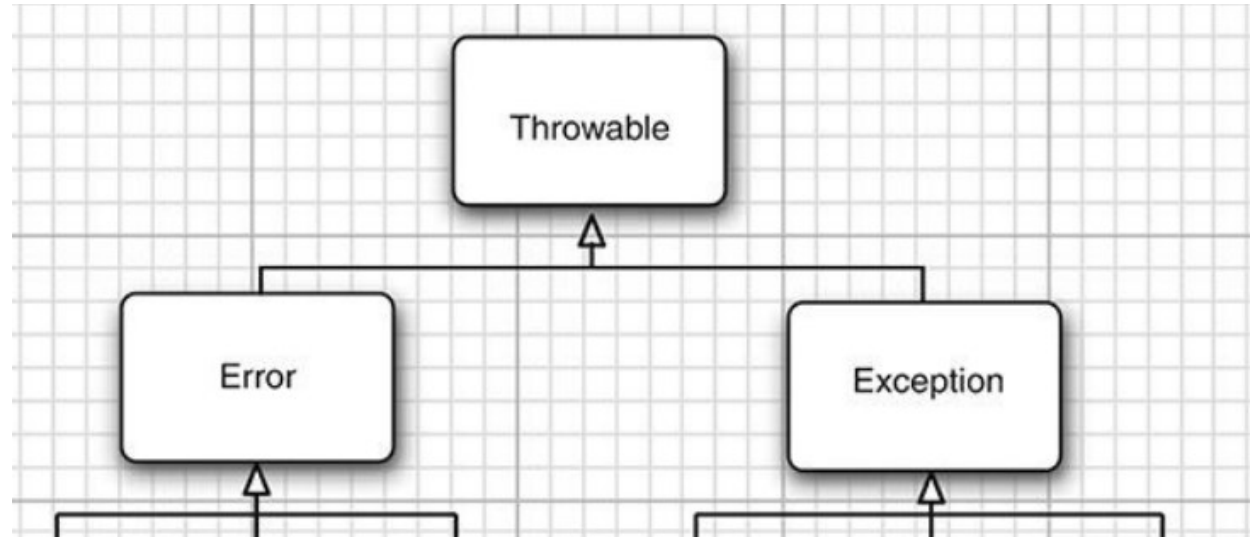
Files are not found
Network breaks down
Memory is not enough
User input is invalid
.....



Java Exception Hierarchy

Only `Throwable` or its subclasses

- Can be thrown by JVM or the `throw` keyword
- Can be caught by the `catch` keyword

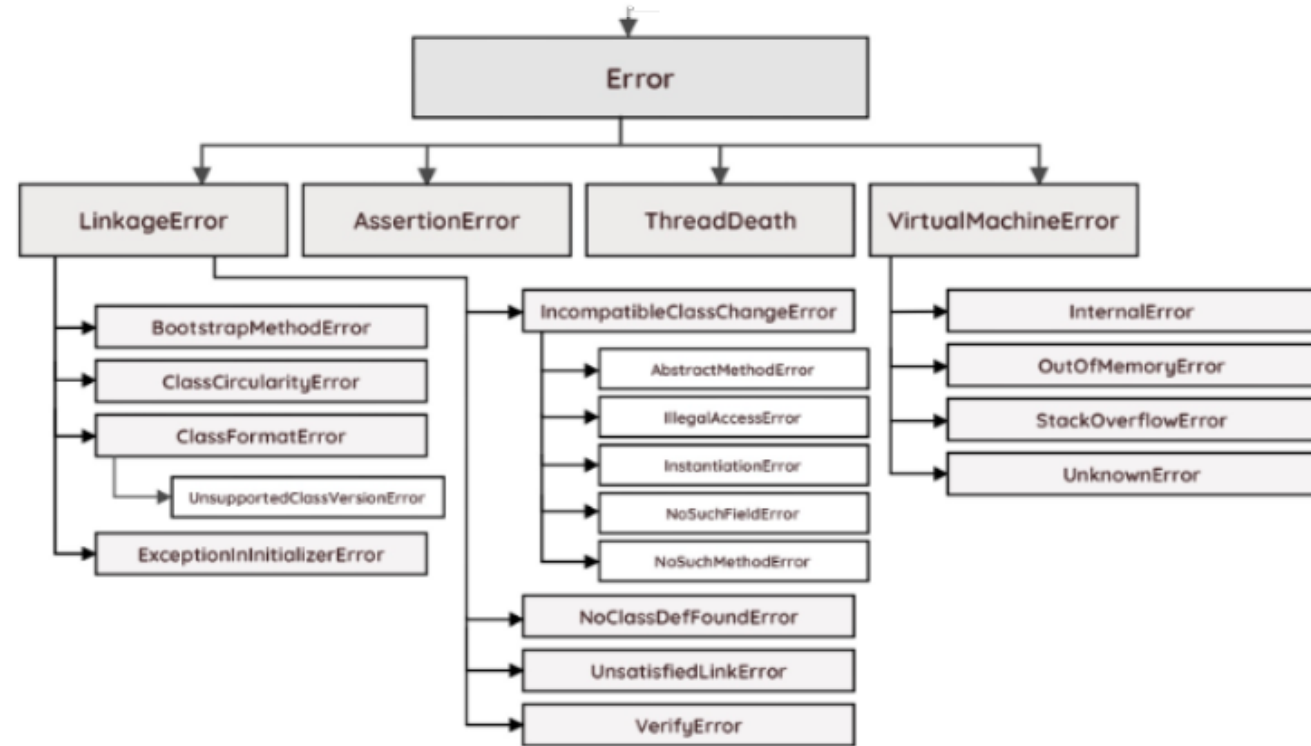


The `Throwable` class is at the top of the Java exception class hierarchy; has two direct subclass: `Error` and `Exception`

Error

- The Error hierarchy describes internal errors and resource exhaustion situations inside the Java runtime system.
- An error indicates serious problems that a reasonable application **should not** try to catch
- E.g., OutOfMemoryError, StackOverflowError

Image source: <https://rollbar.com/blog/java-exceptions-hierarchy-explained>



Mostly thrown by JVM in a scenario considered **fatal**;
No way for the application to recover from that error

Example

- What is the problem with `foo(String s)`?
- Stack is exhausted, leading to `StackOverflowError`
- No recovery during execution, just let it terminate

```
public void foo(String s)
{
    foo(s);
}
```

```
Exception in thread "main" java.lang.StackOverflowError
at examples.foo(examples.java:58)
at examples.foo(examples.java:58)
at examples.foo(examples.java:58)
at examples.foo(examples.java:58)|
at examples.foo(examples.java:58)
at examples.foo(examples.java:58)
at examples.foo(examples.java:58)
```

Exception

- An exception indicates a condition that a reasonable application might **want to** catch.
- RuntimeException and its subclasses are unchecked exceptions
- Others are checked exceptions (think of it as checked by compiler)

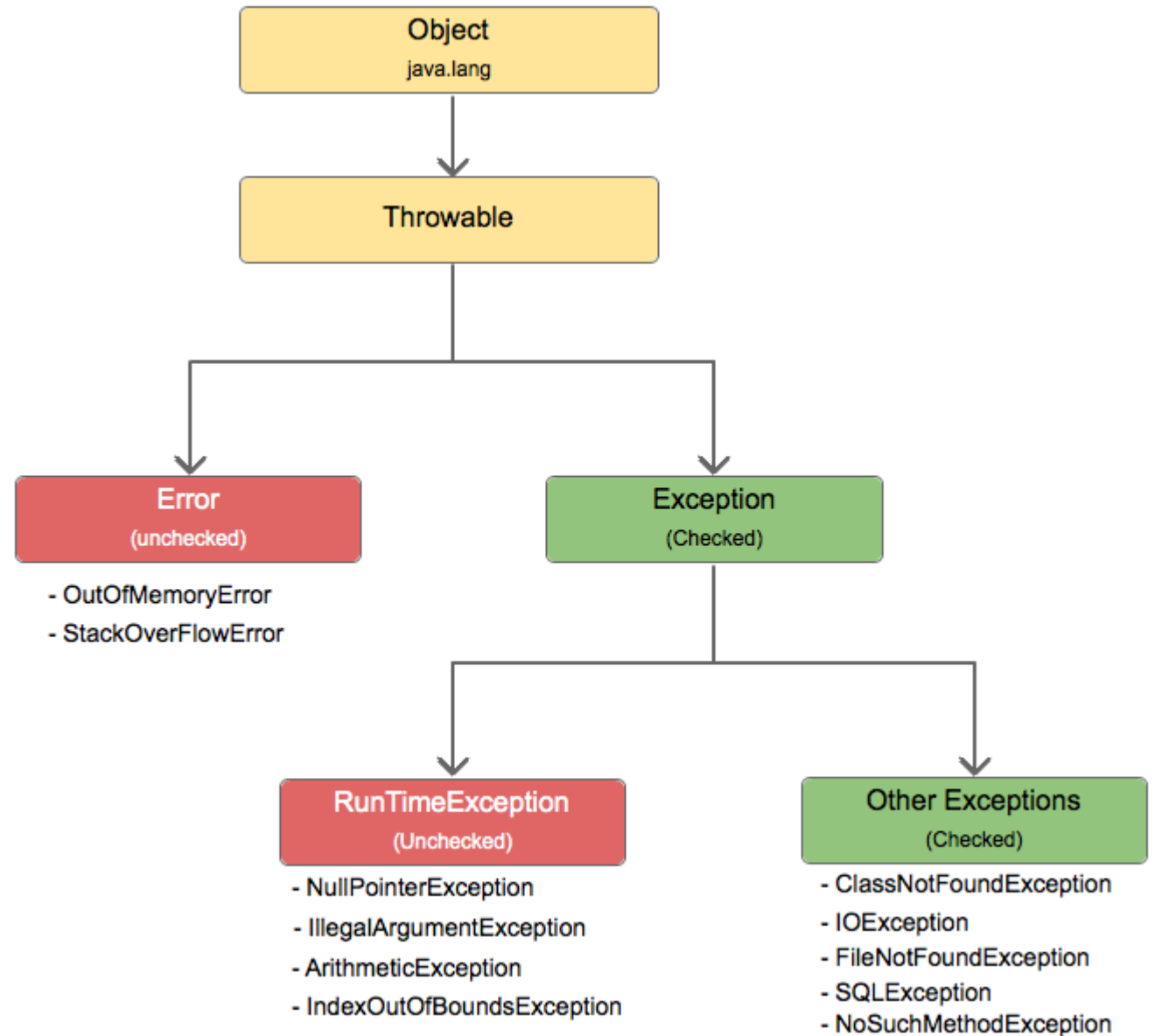


Image source: <https://www.manishsanger.com/tag/exception/>

Checked Exceptions

- Checked Exceptions cannot be ignored at the time of compilation
- Compilers will enforce programmers to handle them
- Two fixes: catch or throw

```
public void processFile() {  
    File text = new File("C:/test.txt");  
    Scanner s = new Scanner(text);  
}
```

Unhandled exception type FileNotFoundException

2 quick fixes available:

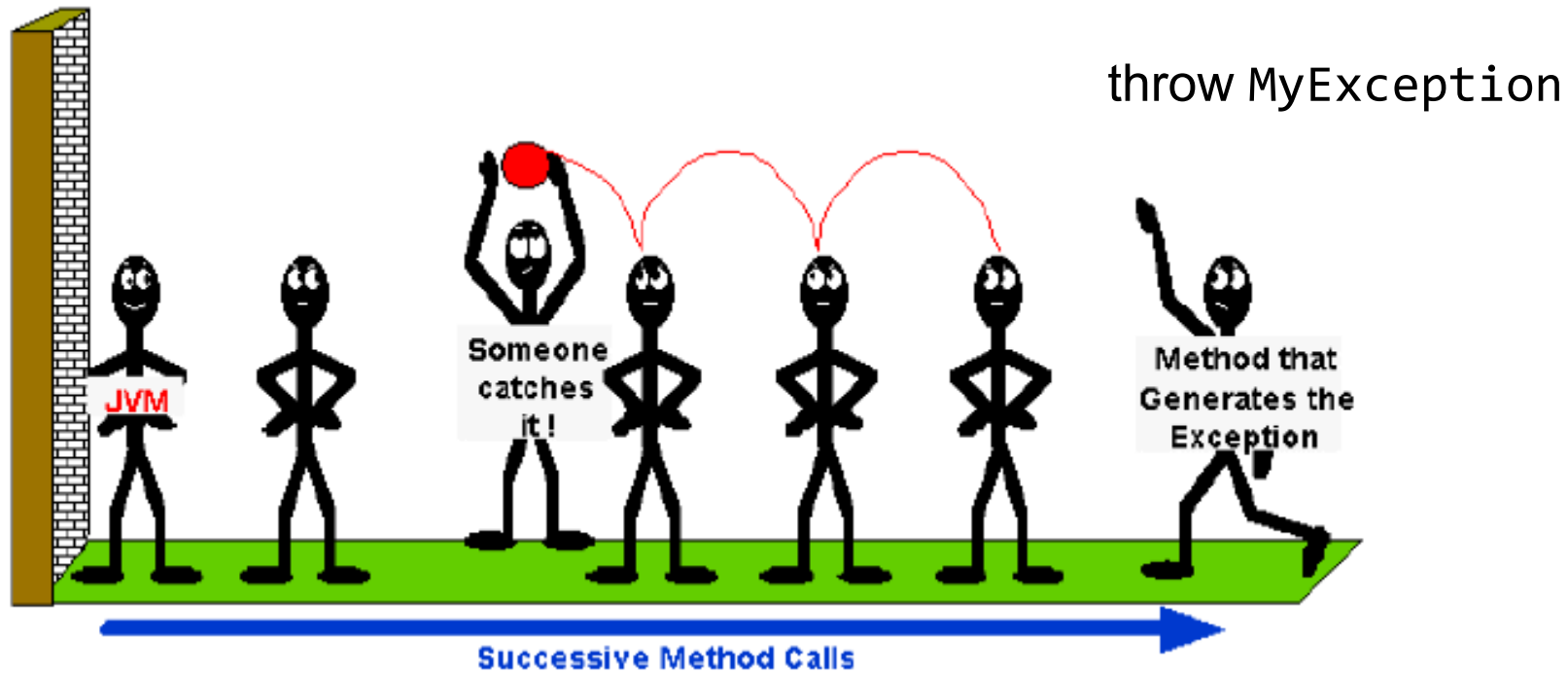
- ! Add throws declaration
- ! Surround with try/catch

Press 'F2' for focus

```
public void processFile() {  
    File text = new File("C:/test.txt");  
    try {  
        Scanner s = new Scanner(text);  
    } catch (FileNotFoundException e) {  
        System.out.println("Cannot find file xxx.");  
    }  
}
```

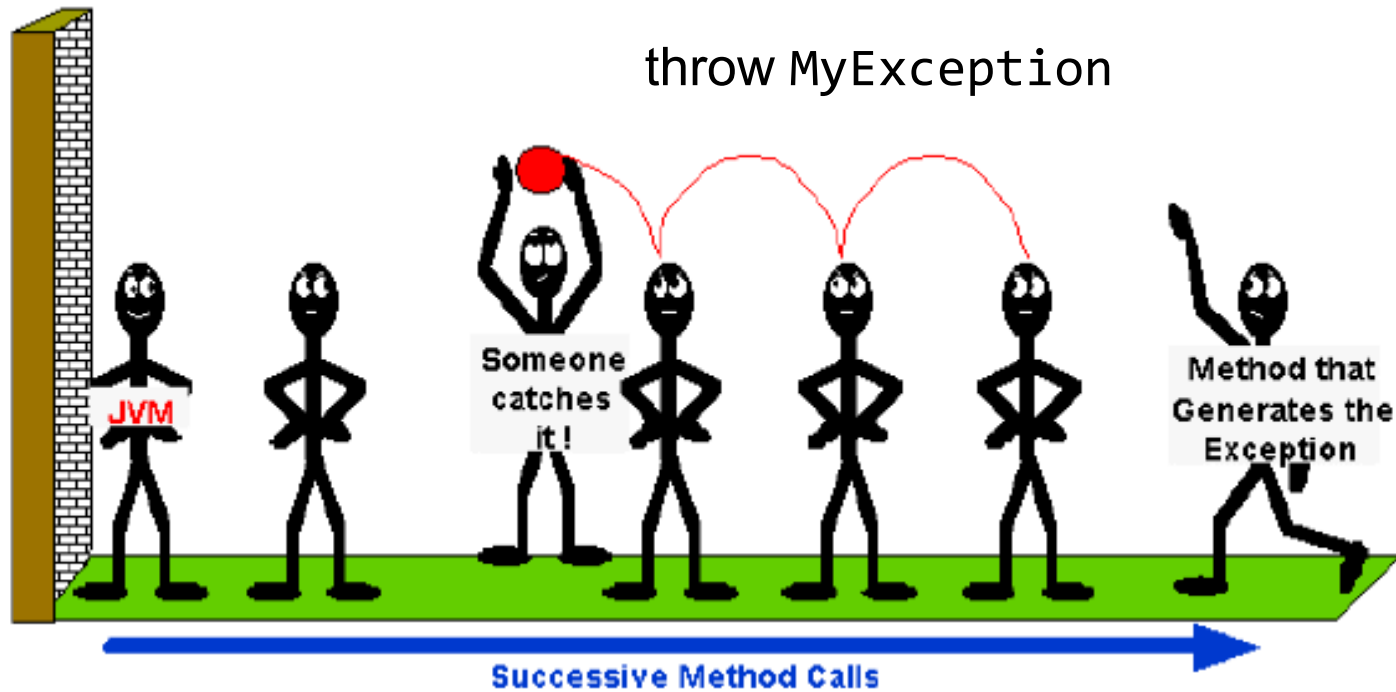
```
public void processFile() throws FileNotFoundException {  
    File text = new File("C:/test.txt");  
    Scanner s = new Scanner(text);  
}
```

Throw & Catch



$A() \rightarrow B() \rightarrow C() \rightarrow D() \rightarrow E() \rightarrow F()$

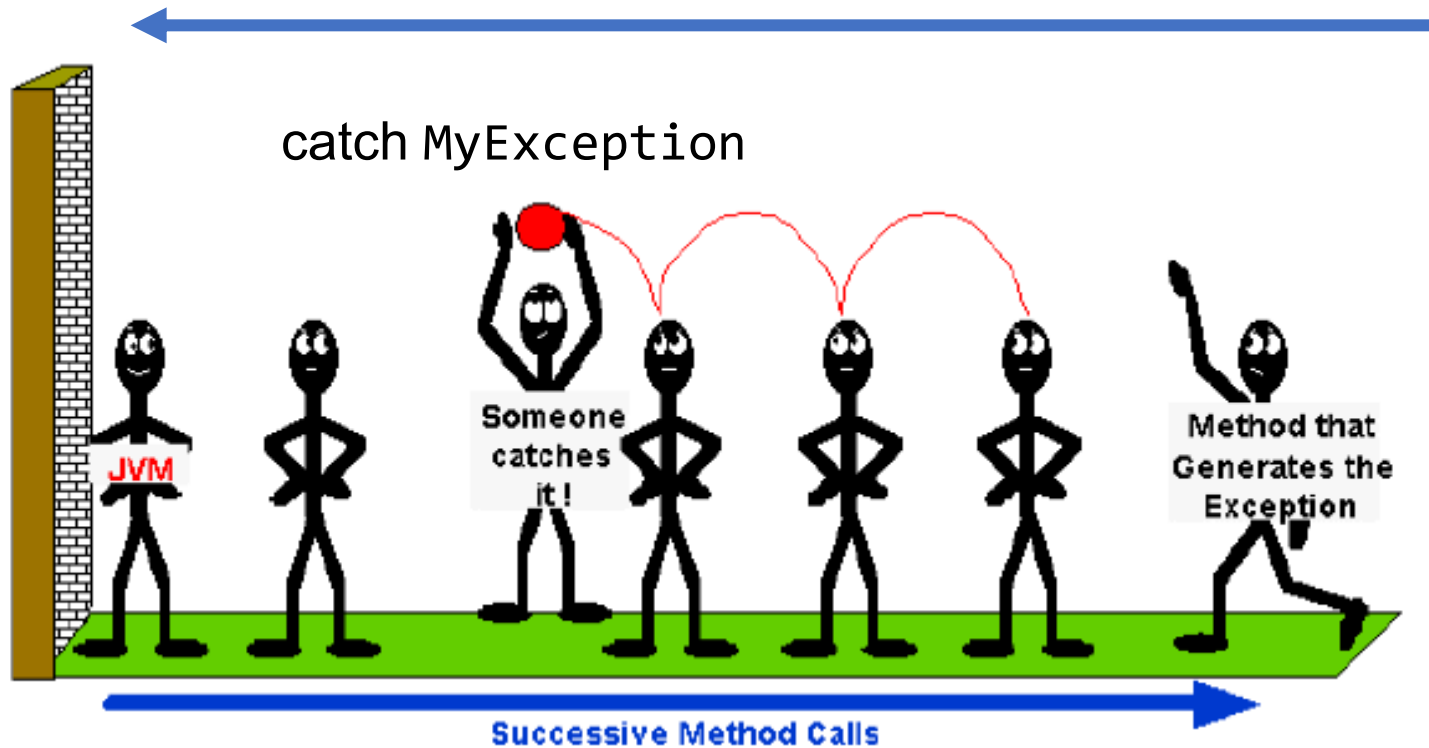
Throw & Catch



$A() \rightarrow B() \rightarrow C() \rightarrow D() \rightarrow E() \rightarrow F()$

Throw & Catch

JVM searches backward through the call stack to find a **matching catch** handler

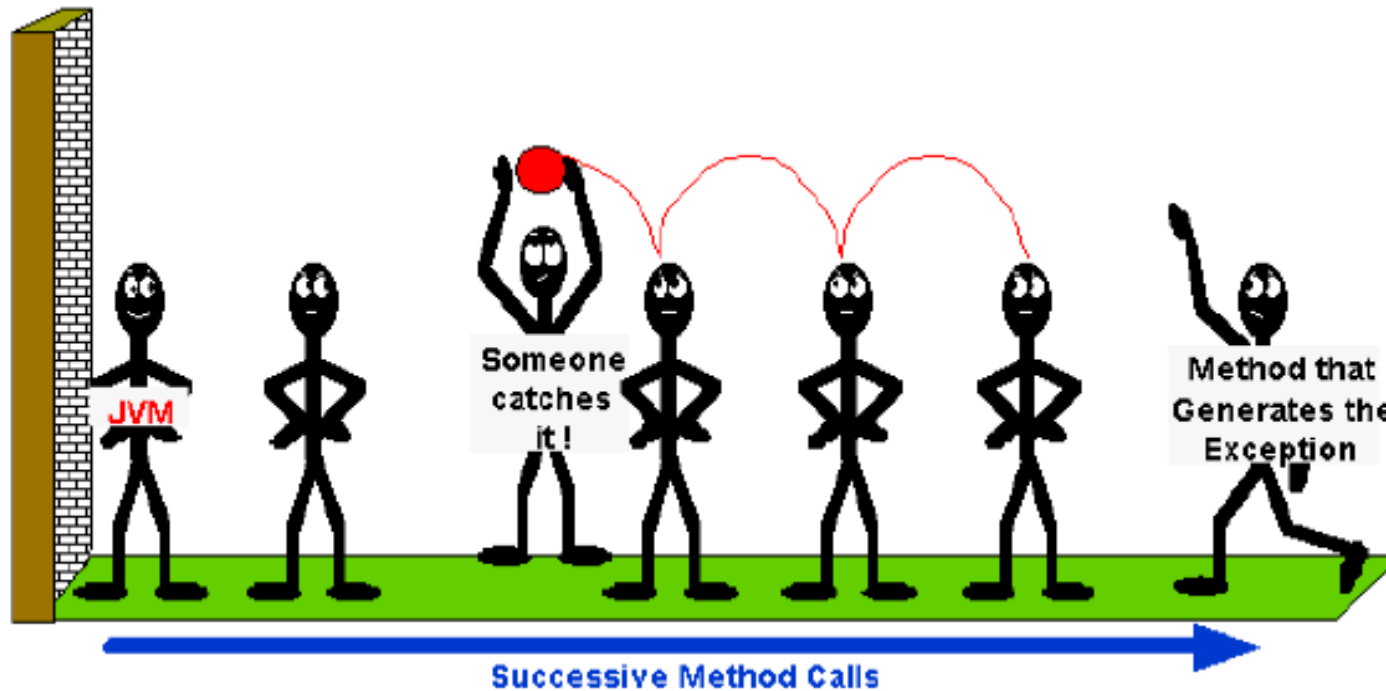


A() → B() → C() → D() → E() → F()

Where to throw, where to catch?

Catch exceptions if you know how to handle them

Throw exceptions that you do not know how to handle



$A() \rightarrow B() \rightarrow C() \rightarrow D() \rightarrow E() \rightarrow F()$

The finally Clause

- When your code throws an exception, it stops processing the remaining code
- Problem occurs if the remaining code does crucial operations such as close resources
- Put such logics in the finally clause, which **always executes**

```
Scanner scanner = null;
try {
    scanner = new Scanner(new File("test.txt"));
    while (scanner.hasNext()) {
        System.out.println(scanner.nextLine());
    }
} catch (FileNotFoundException e) {
    e.printStackTrace();
} finally {
    if (scanner != null) {
        scanner.close();
    }
}
```

try-with-resources

try-with-resources statement ensures that a resource (e.g., `InputStream`, database connection) is **automatically closed** after the program is finished with it

```
open a resource  
try  
{  
    work with the resource  
}  
finally  
{  
    close the resource  
}
```

```
try (Resource res = . . .)  
{  
    work with res  
}
```

try-with-resources

try-with-resources statement ensures that a resource (e.g., `InputStream`, database connection) is **automatically closed** after the program is finished with it

```
try (var in = new Scanner(
    new FileInputStream("/usr/share/dict/words"), StandardCharsets.UTF_8);
    var out = new PrintWriter("out.txt", StandardCharsets.UTF_8))
{
    while (in.hasNext())
        out.println(in.next().toUpperCase());
}
```

No matter how the block exits, both **in** and **out** are closed.

Next Lecture

- Multithreading